

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	SISTEMAS DISTRIBUIDOS				
TÍTULO DE LA PRÁCTICA:	REST vs. RESTful: Diseño e Implementación de Servicios Distribuidos				
NÚMERO DE PRÁCTICA:	06	AÑO LECTIVO:	2026	NRO. SEMESTRE:	2026A
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	X	MÁXIMO DE ESTUDIANTES	5	
FECHA INICIO:	18/05/2026	FECHA FIN:	22/05/2026	DURACIÓN:	2 horas
RECURSOS A UTILIZAR: Entorno de desarrollo: Visual Studio Code o IntelliJ IDEA, Java Development Kit, Maven, Postman. Lenguajes de Programación: Java, Python, Spring Boot, Flask, HTML5 / JavaScript Fetch API					
DOCENTE(s): Mg. Maribel Molina Barriga					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: - Comprender las diferencias conceptuales entre REST y RESTful APIs. - Diseñar servicios distribuidos aplicando principios RESTful. - Implementar APIs cliente-servidor utilizando tecnologías modernas. - Analizar ventajas y limitaciones del enfoque REST en sistemas distribuidos. - Aplicar buenas prácticas de consumo de servicios web.	
TEMAS: REST y RESTful	
COMPETENCIA	C.e. Identifica de forma reflexiva y responsable, necesidades a ser resueltas usando tecnologías de información y/o desarrollo de software en los ámbitos local, nacional o internacional, utilizando técnicas, herramientas, metodologías, estándares y principios de la ingeniería

CONTENIDO DE LA GUÍA

1. ¿Qué es REST?

REST (Representational State Transfer - Transferencia de estado representacional) (La Teoría / El Modelo) es un estilo arquitectónico definido por Roy Fielding para diseñar sistemas distribuidos escalables. Es el conjunto de principios y restricciones arquitectónicas. Define cómo deben comunicarse los sistemas en una red (sin estado, uso de verbos HTTP, arquitectura cliente-servidor). Es un concepto abstracto, una guía de diseño.

Principios:

- **Cliente-servidor:**
Separa la interfaz del usuario del procesamiento y almacenamiento de datos en el servidor.
- **Stateless (sin estado):**
Cada petición contiene toda la información necesaria; el servidor no guarda contexto entre solicitudes.
- **Cacheable:**
Las respuestas pueden almacenarse temporalmente para mejorar rendimiento y reducir tráfico.
- **Uniform Interface (interfaz uniforme):**
Uso de reglas estándar para acceder y manipular recursos (URI, métodos HTTP y formatos comunes).
- **Layered System (sistema por capas):**
La arquitectura puede tener múltiples niveles intermedios sin afectar la interacción cliente-servidor.
- **Code-on-Demand (opcional):**
El servidor puede enviar código ejecutable al cliente para ampliar funcionalidades dinámicamente.

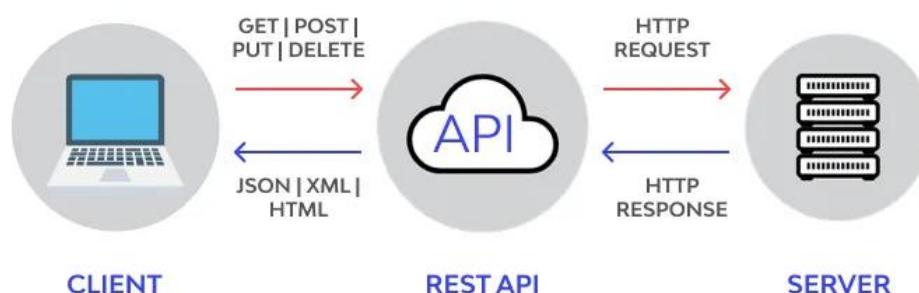
REST: Es la arquitectura que dicta que debes usar peticiones HTTP como `GET` o `POST`.

2. ¿Qué significa RESTful? (La Práctica / La Implementación)

Es el adjetivo que describe a una aplicación, API o servicio web. Cuando dices que una API es RESTful, significa que esta implementación real cumple a cabalidad con las restricciones REST establecidas en la teoría.

RESTful: Es el servicio real (por ejemplo, una API de usuarios) que implementa exactamente esos métodos de forma correcta.

RESTful API



Una API es **RESTful** cuando implementa correctamente las restricciones REST:

REST	RESTful
Modelo conceptual	Implementación concreta
Define principios	Aplica buenas prácticas
Arquitectura	Servicio funcional

Ejemplo:

No RESTful

GET /getUsers

POST /createUser

RESTful

GET /users

POST /users

GET /users/{id}

PUT /users/{id}

DELETE /users/{id}

3. Métodos HTTP

- **GET** → Consultar
- **POST** → Crear
- **PUT** → Actualizar

DELETE → Eliminar**II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE****Caso: API RESTful de Gestión de Productos****Backend (Python + Flask)**

```
from flask import Flask, jsonify, request

app = Flask(__name__)
productos = [
    {"id": 1, "nombre": "Laptop"},
    {"id": 2, "nombre": "Mouse"}
]

@app.route('/productos', methods=['GET'])
def obtener():
    return jsonify(productos)

@app.route('/productos', methods=['POST'])
def agregar():
    data = request.json
    productos.append(data)
    return jsonify({"mensaje": "Producto agregado"}), 201

@app.route('/productos/<int:id>', methods=['DELETE'])
def eliminar(id):
    global productos
    productos = [p for p in productos if p["id"] != id]
    return jsonify({"mensaje": "Producto eliminado"})

if __name__ == '__main__':
    app.run(debug=True)
```

Cliente HTML

```
<!DOCTYPE html>
<html>
<body>
<h2>Lista Productos</h2>
<button onclick="listar()">Consultar</button>
<ul id="lista"></ul>

<script>
async function listar(){
    let r = await fetch("http://127.0.0.1:5000/productos");
    let datos = await r.json();

    let lista = document.getElementById("lista");
    lista.innerHTML="";
```

```
datos.forEach(p=>{
    lista.innerHTML += `<li>${p.nombre}</li>`;
});
}
</script>
</body>
</html>
```

Pruebas con Postman

GET

http://127.0.0.1:5000/productos

POST

```
{
  "id":3,
  "nombre":"Teclado"
}
```

DELETE

/productos/2

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Ejercicio 1: API RESTful Biblioteca (Java + Spring Boot)

Implementar una API para administrar libros.

Debe permitir:

- Listar libros
- Registrar libro
- Buscar por ID
- Eliminar libro

Solución sugerida (Java)

```
@RestController
@RequestMapping("/libros")
public class LibroController {

    List<String> libros = new ArrayList<>();

    @GetMapping
    public List<String> listar(){
        return libros;
    }

    @PostMapping
    public void agregar(@RequestBody String libro){
        libros.add(libro);
    }

    @DeleteMapping("/{id}")
    public void eliminar(@PathVariable int id){
        libros.remove(id);
    }
}
```

Ejercicio 2: API RESTful Estudiantes (Python)

Crear una API que permita:

- Registrar estudiante
- Consultar estudiantes
- Actualizar estudiante
- Eliminar estudiante

Solución sugerida (en python)

```
from flask import Flask, request, jsonify

app = Flask(__name__)
estudiantes=[]

@app.route('/estudiantes', methods=['GET'])
def listar():
    return jsonify(estudiantes)

@app.route('/estudiantes', methods=['POST'])
def agregar():
    estudiantes.append(request.json)
    return jsonify({"ok":True})

@app.route('/estudiantes/<int:i>', methods=['PUT'])
def actualizar(i):
    estudiantes[i]=request.json
    return jsonify({"actualizado":True})

@app.route('/estudiantes/<int:i>', methods=['DELETE'])
def eliminar(i):
    estudiantes.pop(i)
    return jsonify({"eliminado":True})

app.run()
```

Entregables:

- Código fuente (en github)
- Capturas de pruebas
- Evidencia de ejecución correcta del API RESTful y cliente consumidor

III. CUESTIONARIO

1. ¿Por qué una API que utiliza HTTP no necesariamente puede considerarse RESTful?
2. ¿Qué consecuencias tendría diseñar endpoints orientados a acciones y no a recursos en sistemas distribuidos escalables?
3. Compare RESTful frente a RPC y explique en qué escenarios empresariales RESTful podría ser una mala elección.

IV. REFERENCIAS Y BIBLIOGRAFÍA RECOMENDADAS:

- Tanenbaum, A.S. (2008). Sistemas distribuidos: principios y paradigmas. México. Pearson Educación.
- Ceballos, F. J. (2006). Java 2, Curso de programación. México: Alfaomega, RaMa.
- [3]Deitel, H. M., & Deitel, P. J. (2004). Cómo programar en Java. México: Pearson Educación.
- García Tomás, J., Ferrando, S., & Piattini, M. (2001). Redes para procesos distribuidos. México: AlfaomegaRa-Ma.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1
• https://aws.amazon.com/es/what-is/restful-api/ • https://www.ibm.com/es-es/think/topics/rest-apis • https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm • https://spring.io/guides/gs/rest-service • https://flask.palletsprojects.com/en/stable/ • https://learning.postman.com/ • https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API • Fielding, R. (2000). Architectural Styles and the Design of Network-based Software Architectures. • Pautasso, Zimmermann & Leymann (2008). RESTful Web Services vs Big Web Services.		

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN				
TÉCNICAS: <i>Problemas /Ejercicios propuestos</i> <i>/ Preguntas formuladas /</i> <i>Resolución de casos</i>		INSTRUMENTOS: <i>Lista de cotejo, rúbrica.</i>		
CRITERIOS DE EVALUACIÓN				
Criterio	Excelente (5)	Bueno (4)	Regular (3)	Deficiente (1-2)
Comprensión conceptual REST vs RESTful	Diferencia claramente y argumenta técnicamente	Diferencia aceptablemente	Presenta confusión parcial	No diferencia conceptos
Implementación del servicio	Funcional y bien estructurado	Funcional con pequeños errores	Funciona parcialmente	No ejecuta
Consumo cliente-servidor	Correcto y eficiente	Menores errores	Parcial	Incorrecto
Uso correcto de métodos HTTP	Completo	Aceptable	Inconsistente	Incorrecto
Análisis crítico	Reflexión sólida	Adecuada	Superficial	Ausente